Grounded or Guessed: An Execution-Verified Reproducibility Study of LLM-Based Semantic Fault Localization

Doãn Văn An, Tạ Công Nam, Nguyễn Hoàng Hải, Đỗ Xuân Tráng,
Hoàng Trung Kiên, Mai Văn Tùng, Nguyễn Đức Tùng
School of Information and Communication Technology
Hanoi University of Science and Technology, Hanoi, Vietnam

***Abstract*—LLM-based fault localizers increasingly claim to ground the model's reasoning in program execution. We test that claim on SemLoc, a recent localizer that turns LLM-inferred semantic constraints into a violation spectrum. We recompute every published configuration from the authors' released logs, audit the evaluation, re-derive the benchmark's fault locations by execution, rerun the pipeline three times with a current model, and measure how much of the accuracy is due to execution evidence and how much to the LLM's own guess. Four findings stand out. First, top-1 accuracy is guessed by design: the released code adds an undocumented bonus that always places the LLM's own fault-line guess first, so SemLoc's Acc@1 (52.3%) is the guess's; the constraint spectrum alone would reach 43.2%, and its value lies at top three, where it lifts accuracy from 52.3% to 75.4%. Second, on faults that a failing test reaches, a single prompt showing the LLM the failing tests and per-line coverage beats the whole pipeline by +16.2 Acc@3 points at about a fifth of the tokens, with no detectable change under identifier renaming; over all original labels, which include faults no test reaches, the gain shrinks to +7.8 points. Third, published numbers depend on scoring choices and benchmark labels: 56 of 250 fault lines are blank, and the released counterfactual step never changes a ranking. Fourth, line-level accuracy is ambiguous for faults spanning a guard and the statement it guards, which moves Acc@1 by over twenty points without changing the order of methods. Code, data, LLM logs and a replayable cache are public.**

I. Introduction

Fault localization tells a developer where a failing program is wrong. Spectrum-based fault localization (SBFL) [1], [2], [3] ranks statements by how often failing and passing tests execute them; it is cheap and needs no model of the program. It breaks down on *semantic* faults, where the program takes the right path but computes a wrong value: failing and passing tests then execute the same statements and the spectrum collapses. On the SemFault-250 benchmark, Ochiai places the fault first in only 4.0% of programs under fair scoring.

Large language models (LLMs) attack the problem from the other side: they read the code and guess the faulty line. Systems such as AutoFL [4], AgentFL [5] and FlexFL [6] report strong results, but an LLM's answer is stochastic, hard to verify, and possibly memorized from the benchmark [7], [8]. SemLoc [9] proposes a middle path. It asks the LLM for *executable* semantic constraints, instruments the program with them, runs the tests, and ranks lines by a constraint-violation spectrum; a counterfactual step then patches suspicious lines and re-runs the tests. The promise is that execution, not the model, has the final word. SemLoc reports 42.8% Acc@1 on SemFault-250, far above SBFL.

This paper asks whether that accuracy is grounded or guessed. If execution evidence is what makes SemLoc work, removing it should hurt and adding more of it should help; if the model's own guess does the work, the pipeline adds cost without adding evidence. Answering requires more than re-running the tool: one must check how results are scored, whether the benchmark's fault locations are right, how much each signal contributes, and whether conclusions survive real bugs, renamed code and other models. We study three research questions: **RQ1** (reproduction and evaluation audit), **RQ2** (improvement and a simple baseline), and **RQ3** (robustness on BugsInPy [10], identifier-renamed programs and five model families).

Our study uses execution as the arbiter throughout. We re-derive SemFault-250's fault locations with a minimal-fix oracle in which an LLM only proposes one-line patches and the tests decide; we measure the contribution of SemLoc's two signals, the constraint spectrum and a bonus the released code adds to the LLM's own fault-line guess; and we compare SemLoc with *LLM + coverage*, a single prompt that shows the model the failing tests and how many failing and passing tests execute each line. Every LLM configuration runs three times; differences come with program- and template-cluster bootstrap confidence intervals.

The answer to the title is: top-1 is guessed, top-3 is grounded. The bonus (1.2) exceeds any spectrum score (at most one), so SemLoc's first line is always the LLM's guess: its Acc@1 of 52.3% is the guess's, and the spectrum alone would reach 43.2%. The spectrum earns its keep below the first line, lifting Acc@3 from the guess's 52.3% to 75.4%. Execution evidence works best when handed to the model directly: on faults a failing test reaches, LLM + coverage gains +16.2 Acc@3 points over SemLoc on the original labels [+9.4, +22.8].

Contributions.

- A *grounded-vs-guessed* analysis of LLM-based fault lo-

calization: SemLoc's top-1 line is, by an undocumented design choice, the LLM's own guess; its execution evidence contributes at top three. A Shapley split and the marginal values of both signals quantify this on our runs and on the authors' logs.

- Evidence that *a single prompt with test outcomes and per-line coverage beats the pipeline* on reached faults (+16.2 Acc@3 on original labels, +16.6 on repaired ones) at 2.6k instead of 14.0k tokens per program, with no detectable change under renaming; over all original labels the gain is +7.8 and SemLoc is better at Acc@5.
- An *evaluation audit*: the undocumented guess bonus; a counterfactual step that never changes a ranking; a released scoring script that credits delta debugging with 38.0% where the paper reports zero; two bugs in the released BugsInPy runner; and a re-execution with the paper's own model that falls short of the logs.
- *SemFault-250-repaired*: 82 fault locations confirmed and 46 corrected by execution, and the finding that line-level Acc@1 shifts by more than twenty points under the guard/guarded-statement ambiguity.
- CovGate, a small coverage tie-breaker on top of SemLoc, an evaluation checklist, and a fully replayable artifact.

Code, data and LLM logs: https://github.com/doanan293/grounded-or-guessed.

II. Related Work and Research Direction

Spectrum- and mutation-based localization. Tarantula [1], Ochiai [2] and DStar [11] score statements from coverage; surveys [3], [12] note two evaluation pitfalls we revisit: tie handling and omission faults, which have no faulty line to blame. Mutation-based localization [13], [14] replaces coverage with mutant behavior; delta debugging [15] and cause-effect chains [16] underlie counterfactual reasoning; and Daikon [17] infers likely invariants from traces, whereas SemLoc asks an LLM for them.

LLM-based localization. AutoFL [4] lets an LLM navigate code with tool calls at method level; AgentFL [5] scales this to projects with agents; FlexFL [6] feeds SBFL output to open-source LLMs, the closest idea to our coverage baseline but at method level on Java. An industrial study of crash reports [18] finds that mined rankings in the prompt help LLMs localize, and EffiHolmes [19] contrasts two executions (normal vs. scaled workload) before LLM reasoning for performance faults. SemLoc [9] is the first to rank lines by violations of LLM-inferred executable constraints. None of these works measures how much of the accuracy comes from execution evidence rather than from the model's own guess.

Benchmarks and validity. Defects4J [20], BugsInPy [10] and QuixBugs [21] provide faults with triggering tests. Pearson et al. [22] show that results on artificial faults do not transfer to real ones and that tie and omission handling can reorder techniques. For LLMs, threats include closed-model drift and benchmark contamination [7], [8], [23], mitigated by rewriting or renaming.

Direction. We combine these threads: we audit scoring as Pearson et al. recommend, verify the benchmark by execution, test memorization by renaming, and add the missing measurement—the share of accuracy that is grounded.

III. Background

SBFL and metrics. For a line s, let e_f, e_p be the numbers of failing and passing tests that execute it and n_f the failing tests that do not. Ochiai scores s as $e_f/\sqrt{(e_f+n_f)(e_f+e_p)}$; lines with $e_f = 0$ are unranked. The rank of a program's fault is the best rank of any ground-truth line, and Acc@k is the fraction of programs whose fault ranks within k. We break ties by the *average* rank of the tied group and keep the denominator fixed: a program a tool fails on counts as a miss.

Semantic faults. We label a fault *semantic* when some faulty line is executed by at least one failing and one passing test (coverage cannot separate it), *coverage-revealed* when only failing tests reach it, and *not executed* when no failing test reaches any ground-truth line. Labels are recomputed for each ground-truth variant.

SemLoc. Fig. 1 summarizes SemLoc [9]. One LLM call returns constraints in a closed intermediate language, each attached to a program point; instrumentation evaluates them while the tests run; each constraint gets an Ochiai score over the constraint-by-test violation matrix, and a line gets the maximum score of the constraints attributed to it. Counterfactual verification drops constraints violated on passing tests, asks the LLM for a one-line patch at each remaining constraint's line, and re-runs the tests. The released code also adds 1.2 to the line the LLM names as its own fault-line guess—more than the maximum constraint score of one—which the paper does not describe.

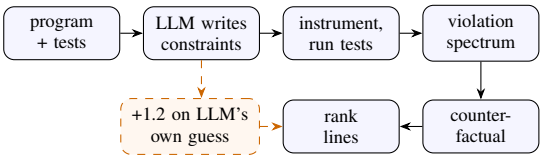


Fig. 1. SemLoc's pipeline [9]; the dashed box is the undocumented guess bonus found in the code.

IV. Approach

A. Evaluation audit

From the authors' logs we recompute every published configuration under two scorings: the *authors' scoring* as implemented in the released script (line-order tie breaking, any delta-debugging hit counted as rank one, programs that fail to run dropped from the denominator) and our *fair scoring* (average-rank ties, fixed denominator). We then remove the undocumented guess bonus, check whether the counterfactual step changes any ranking, and re-execute SemLoc with the paper's own model (Claude Sonnet 4.6) on the dev split.

B. SemFault-250-repaired

56 of the 250 ground-truth lines are blank and 65 programs have no failing test reaching the fault line. We re-derive fault locations with a *minimal-fix oracle*: a line is a fault location if replacing it (or filling a blank line, for omissions) with a small edit makes every test pass. An edit is small if its token-level edit distance to the original line (tokens are identifiers, numbers and single punctuation characters) is at most two or at most a third of the line's tokens. For each program: (1) for each ground-truth line, an LLM proposes candidate edits (GPT-6 luna, two attempts at $T = 0.8$, then up to four; a second pass with Claude Sonnet 5.5 for lines still unverified) and the tests decide; (2) a ground-truth line that admits a passing small edit is *confirmed*; (3) only if no failing test reaches the original line, lines within four of it are searched and the closest line with the smallest passing edit replaces it (*corrected*); (4) otherwise the line stays *unverified*. The oracle confirms 82 locations and corrects 46; 111 reachable lines keep their original labels (no proposed edit passed), and 11 programs cannot be verified and are dropped. Fixes written onto blank lines name no rankable statement and are also dropped, leaving 226 programs. Because executed labels are never searched for alternatives, the oracle cannot detect two equivalent fix locations (Section V). We report every result on both the original and the repaired labels.

C. Grounded-vs-guessed decomposition

SemLoc's line score is the spectrum score plus a bonus of 1.2 on the LLM's guessed line. We define a two-player game {guess, spectrum}: $v(S)$ is Acc@k when lines are ranked by the signals in S only ($v(\emptyset) = 0$; "guess only" ranks the guessed line alone; "spectrum only" ranks by spectrum score with average-rank ties); with both, the scores equal SemLoc as released. We report the marginal values $v(\{\text{guess}\})$, $v(\{\text{spectrum}\})$ and $v(\{\text{guess}, \text{spectrum}\})$ together with the Shapley split. Because Acc@k is a mean of per-program hits, the aggregate Shapley value is the mean of per-program values, which lets us bootstrap confidence intervals by program and by template cluster. An extended game adds raw Ochiai (added with weight one) and counterfactual re-ranking as two more players; its grand coalition is a different system from SemLoc.

D. CovGate

CovGate post-processes SemLoc's line scores with coverage. Writing $\sigma(s)$ for the spectrum score, g for the guessed line and F for the lines executed by some failing test,

$$c(s) = \mathbb{1}[s \in F]^{\gamma} \left(\sigma(s) + \beta \mathbb{1}[s = g] + \lambda \,\mathrm{Ochiai}(s)\right), \quad (1)$$

where $\gamma \in \{0, 1\}$ switches a *coverage gate*, $\lambda \in \{0; 0.01; 0.1; 0.5\}$ weights Ochiai, and the bonus β is either always 1.2 or, in the *grounded* variant, kept only if a failing test executes g and a constraint violated in some failing test is attributed to g. A fourth switch, *counterfactual re-ranking*, promotes the lines of constraints that the counterfactual patches confirm (computed but unused in the released code). Parameters are chosen on the 50-program dev split only, by Acc@3 then Acc@1, pooled over five published configurations. The selected configuration has no gate and the original bonus, with λ = 0.01 on the original and 0.5 on the repaired labels; CovGate as evaluated is thus an Ochiai tie-breaker.

E. Baselines

LLM direct is SemLoc's own direct-prediction mode: one call with the program only, asking for the faulty lines. *LLM + coverage* is ours: one call at $T = 0.8$ whose prompt lists the program with every line annotated `[f=`e_f`/p=`e_p`]`, states the numbers of failing and passing tests, names each failing test with its first error line, and asks for the five most suspicious lines as JSON. Compared with LLM direct it adds test outcomes and error messages as well as coverage, so the comparison measures a single prompt with execution evidence against the pipeline, not coverage in isolation. The prompt follows the finding that mined rankings help LLMs [18] and FlexFL's SBFL hints [6]; it was fixed before looking at test results.

V. Evaluation

A. Setup

SemFault-250 [9] has 250 single-function Python programs with one injected fault and about ten tests each. We fix a 50/200 dev/test split stratified by fault label and group structurally similar programs into template clusters (4-gram Jaccard similarity of at least four fifths on normalized code). Real bugs are the 31 youtube-dl bugs of BugsInPy [10] whose environment builds, whose triggering tests fail before and pass after the fix, and whose coverage can be measured; the target function is the one containing a developer-changed line that a declared failing test executes, and its changed lines are the ground truth. The renamed variant replaces every program-defined identifier with an opaque name and blanks comments

TABLE I
PUBLISHED CONFIGURATIONS UNDER THE AUTHORS' AND FAIR SCORING (RELEASED LOGS, 250 PROGRAMS)

	Author scoring			Fair scoring		
Configuration (all 250 programs)	Acc@1	Acc@3	Acc@5	Acc@1	Acc@3	Acc@5
SemLoc, Claude Sonnet 4.6, T=0.8	42.8	68.0	76.8	42.8	65.6	76.8
SemLoc, Gemini 2.5 Pro, T=0.8	39.8	64.7	73.5	39.6	64.4	72.8
SemLoc Claude T=0.8 without LLM-guess bonus	–	–	–	30.0	55.6	66.4
SemLoc Gemini T=0.8 without LLM-guess bonus	–	–	–	27.2	55.6	64.0
SemLoc w/o CF (separate run), Claude	30.8	59.6	68.8	–	–	–
LLM direct, Claude	35.2	41.2	41.2	35.2	41.2	41.2
SBFL Ochiai	6.4	13.2	30.4	4.0	16.0	48.0
SBFL DStar (recomputed)	–	–	–	4.0	16.0	48.0
Delta debugging	38.0	38.0	38.0	0.0	0.0	0.0

while preserving line numbers and each test's outcome (199 test programs). The main model is Gemini 3.8 Flash; the model-family study adds GPT-6 luna, Claude Sonnet 5.5, gpt-oss-120b and Gemini 3.5 Flash-Lite on a fixed subset of the test split (44 programs with repaired labels). All models run at their provider-default reasoning level; SemLoc and LLM + coverage run at $T = 0.8$. Every LLM configuration is repeated three times; a run is used only if it finished without unrecovered LLM errors. Differences are Δ (percentage points) with 95% bootstrap CIs that resample template clusters (10000 draws) after averaging each program over repeats; we also report each repeat's McNemar test and, in Table II, a matched-pairs rank-biserial effect size on reciprocal rank. We report about twenty such comparisons without multiplicity correction and read single intervals near zero with caution.

B. RQ1: Reproduction and evaluation audit

Recomputing from the released logs reproduces all 13 published configurations exactly. Re-executing SemLoc with the paper's own model on the dev split does not: Acc@1 32.0% and Acc@3 44.0% against 36.0% and 68.0% in the logs for the same programs; the re-executed constraints are violated in failing tests less often, which we could not trace to our infrastructure.

Table I shows how scoring matters. The released scoring script credits delta debugging with 38.0% (any hit counts as rank one), although the paper reports zero and fair scoring gives 0.0%, since it ranks no line; SBFL's Acc@5 rises from 30.4% to 48.0% under average-rank ties. SemLoc is stable under fair scoring, but without the undocumented guess bonus its Acc@1 falls from 42.8% to 30.0% (Claude) and from 39.6% to 27.2% (Gemini). In the released code the counterfactual step never changes a ranking (0 of 250 programs in all five configurations with data). The paper's w/o-counterfactual figure comes from a separate run whose scores were not released; its Acc@1 (30.8%) is close to SemLoc without the bonus, which suggests a configuration difference rather than an effect of the step, but we cannot determine which. The released BugsInPy runner reports the rank of the LLM's own guess rather than of the fixed lines, and its patch-and-restore step leaves stale bytecode that can hide the bug in the next run; we fixed the latter in our driver.

Labels. Repairing the ground truth raises SemLoc's Acc@1 on our runs from 36.8% (200 programs) to 52.3% (179 test programs); part of this is the smaller denominator, since the repaired set drops faults that cannot be verified. On the original labels restricted to faults a failing test reaches, SemLoc already scores 49.8%.

C. RQ2: Improvement and a simple baseline

Single prompt vs. pipeline. Table II compares methods on the test split. The comparison depends on whether the fault is reachable. On faults that a failing test reaches, LLM + coverage beats SemLoc at Acc@3 by +16.2 points [+9.4, +22.8] on the original labels (148 programs) and by +16.6 [+10.5, +22.8] on the repaired ones; it also wins on purely semantic faults, where coverage alone cannot separate the fault (Acc@3 91.8% vs. 77.6%, Table III). Over all original labels, which include faults no failing test reaches, the gain shrinks to +7.8 [+0.2, +15.2] and SemLoc is better at Acc@5 (75.0% vs. 70.0%): a prompt built on coverage cannot point at lines no test executes. At Acc@1 the gain (+4.8 [+1.5, +8.7]) is visible only when repeats are pooled; single repeats do not reach significance. LLM direct, which sees the program only, is not worse than SemLoc at Acc@1 (Δ +4.5, CI includes zero) but changes Acc@3 by -12.7 points. Because LLM + coverage adds test outcomes, error messages and coverage at once, the gap over LLM direct measures execution evidence in the prompt as a whole, not coverage alone.

CovGate. The selected CovGate (an Ochiai tie-breaker, Section IV) changes Acc@3 by +3.9 [+1.6, +6.8] on the repaired labels and by +1.0 [+0.2, +2.3] on the original ones, and leaves Acc@1 unchanged. The coverage gate alone gives a similar gain on the repaired labels but changes Acc@3 by -2.7 on the original ones, because it removes every fault no failing test reaches. Keeping the bonus only when execution supports the guess changes Acc@1 by -4.1, and counterfactual re-ranking by -2.4: execution evidence does not separate right guesses from wrong ones.

Grounded or guessed. Table IV gives the Shapley split; the marginal values are more telling. Because the bonus exceeds any spectrum score, adding the spectrum to the guess cannot change the first line: guess only, and SemLoc, both reach 52.3% Acc@1, while spectrum only reaches 43.2%. The Shapley value of the spectrum at Acc@1 (21.6) is therefore credit for what the spectrum would achieve without the guess, not a contribution to SemLoc's top line. At Acc@3 the spectrum matters: the guess names one line and stays at 52.3%, spectrum only reaches 70.8%, and together they reach 75.4%. On the authors' own logs (Claude) the split is 42.8 = 27.8 + 15.0.

Cost. SemLoc spends 14.0k tokens per program, and its counterfactual step accounts for 42% of its LLM calls; LLM + coverage spends 2.6k. Fig. 2 places five models on the same subset: for every model LLM + coverage has higher Acc@3 at lower cost than SemLoc. These per-model comparisons have no CIs; at Acc@1 GPT-6 luna does not improve, and on this subset LLM direct has the highest Acc@1 for Gemini 3.8 Flash (59.9% vs. 56.1%).

TABLE II

RQ2 ON THE TEST SPLIT (GEMINI 3.8 FLASH, THREE REPEATS; Δ VS. SEMLOC WITH 95% CLUSTER-BOOTSTRAP CI; r_{rb}: MATCHED-PAIRS RANK-BISERIAL ON RECIPROCAL RANK)

Method	Acc@1	Acc@3	Acc@5	ΔAcc@1 [95% CI]	ΔAcc@3 [95% CI]	r_{rb}
Original ground truth, $n = 200$, 3 repeats						
SemLoc (as released)	36.8±0.8	60.3±2.3	75.0±0.9	–	–	–
w/o LLM-guess bonus	29.2±1.4	57.7±2.5	72.7±1.3	-7.7 [-13.9, -1.6]	-2.7 [-4.9, -0.9]	-0.38
bonus only when grounded	34.7±1.5	58.2±2.9	72.8±1.5	-2.2 [-4.2, -0.6]	-2.2 [-4.2, -0.6]	-0.94
+ coverage gate only	36.8±0.8	57.7±1.8	62.5±1.3	+0.0 [0.0, 0.0]	-2.7 [-7.7, 1.7]	-0.59
+ λ·Ochiai only	36.8±0.8	61.3±1.9	75.0±0.9	+0.0 [0.0, 0.0]	+1.0 [0.2, 2.3]	+0.54
+ CF re-ranking	35.0±1.0	62.2±2.5	77.7±1.8	-1.8 [-3.8, -0.3]	+1.8 [0.5, 3.5]	+0.36
CovGate (ours)	36.8±0.8	61.3±1.9	75.0±0.9	+0.0 [0.0, 0.0]	+1.0 [0.2, 2.3]	+0.54
CovGate + CF re-ranking	35.0±1.0	62.5±2.2	77.7±1.8	-1.8 [-3.8, -0.3]	+2.2 [0.9, 3.8]	+0.46
LLM direct (1 prompt)	40.0±0.0	43.0±0.5	43.2±0.8	+3.2 [-0.8, 7.4]	-17.3 [-23.3, -11.1]	-0.51
LLM + coverage (1 prompt, ours)	40.2±0.6	68.2±0.3	70.0±0.9	+3.3 [0.5, 6.5]	+7.8 [0.2, 15.2]	+0.04
SBFL Ochiai	3.0±0.0	15.5±0.0	48.0±0.0	-33.8 [-41.2, -26.8]	-44.8 [-52.8, -36.6]	-0.87
Execution-repaired ground truth, $n = 179$, 3 repeats						
SemLoc (as released)	52.3±0.9	75.4±2.0	82.7±1.1	–	–	–
w/o LLM-guess bonus	43.2±2.1	70.8±2.3	78.4±1.7	-9.1 [-16.7, -1.9]	-4.7 [-7.9, -2.1]	-0.37
bonus only when grounded	48.2±1.7	71.3±2.8	78.6±2.0	-4.1 [-7.2, -1.7]	-4.1 [-7.2, -1.7]	-1.00
+ coverage gate only	52.3±0.9	79.1±2.0	85.1±1.3	+0.0 [0.0, 0.0]	+3.7 [1.5, 6.6]	+1.00
+ λ·Ochiai only	52.3±0.9	79.3±1.7	85.1±1.3	+0.0 [0.0, 0.0]	+3.9 [1.6, 6.8]	+0.85
+ CF re-ranking	49.9±0.9	76.2±1.8	83.4±1.4	-2.4 [-4.8, -0.6]	+0.7 [0.0, 2.1]	-0.35
CovGate (ours)	52.3±0.9	79.3±1.7	85.1±1.3	+0.0 [0.0, 0.0]	+3.9 [1.6, 6.8]	+0.85
CovGate + CF re-ranking	49.9±0.9	77.1±2.2	85.3±1.2	-2.4 [-4.8, -0.6]	+1.7 [0.5, 3.3]	+0.28
LLM direct (1 prompt)	56.8±0.7	62.8±0.7	62.9±0.9	+4.5 [-0.4, 9.7]	-12.7 [-19.7, -6.2]	-0.28
LLM + coverage (1 prompt, ours)	57.2±0.9	92.0±0.3	93.7±0.9	+4.8 [1.5, 8.7]	+16.6 [10.5, 22.8]	+0.65
SBFL Ochiai	3.9±0.0	19.0±0.0	67.6±0.0	-48.4 [-58.6, -38.3]	-56.4 [-67.0, -45.4]	-0.85

TABLE III

RQ2 BY FAULT LABEL (LABELS OF THE REPAIRED GROUND TRUTH)

		SBFL		SemLoc		CovGate		LLM + cov.	
Fault label	n	Acc@1	Acc@3	Acc@1	Acc@3	Acc@1	Acc@3	Acc@1	Acc@3
Semantic	162	1.8	17.3	54.7	77.6	54.7	81.1	60.1	91.8
Coverage-revealed	17	23.5	35.3	29.4	54.9	29.4	62.7	29.4	94.1
Executable (all but not executed)	179	3.9	19.0	52.3	75.4	52.3	79.3	57.2	92.0

TABLE IV

SHAPLEY SPLIT OF SEMLOC'S ACCURACY (OUR RUNS, THREE REPEATS; TEMPLATE-CLUSTER CIS). UPPER BLOCK: THE GAME WHOSE TOTAL IS SEMLOC AS RELEASED; LOWER BLOCK: ALL FOUR SIGNALS (A DIFFERENT SYSTEM).

Ground truth	Metric	Total	LLM guess	Semantic spectrum	Coverage	CF
SemLoc as released (guess + spectrum)						
Original ($n = 200$)	Acc@1	36.8	22.2 [16.5, 28.4]	14.6 [11.8, 17.5]	–	–
Original ($n = 200$)	Acc@3	60.3	19.8 [16.1, 23.8]	40.6 [35.6, 45.3]	–	–
Repaired ($n = 179$)	Acc@1	52.3	30.7 [23.5, 37.8]	21.6 [17.4, 26.1]	–	–
Repaired ($n = 179$)	Acc@3	75.4	28.5 [23.4, 33.5]	46.9 [41.9, 51.7]	–	–
All signals (+ raw Ochiai, + CF re-ranking)						
Original ($n = 200$)	Acc@1	35.0	22.7 [17.4, 28.3]	12.5 [10.0, 15.2]	-0.5 [-1.5, 0.6]	0.3 [-0.6, 1.1]
Original ($n = 200$)	Acc@3	59.0	17.6 [14.2, 21.4]	35.8 [31.6, 40.2]	4.6 [0.1, 9.0]	1.1 [0.5, 1.8]
Repaired ($n = 179$)	Acc@1	49.9	31.9 [25.3, 38.5]	18.6 [14.4, 22.9]	-1.1 [-2.3, 0.2]	0.5 [-0.7, 1.5]
Repaired ($n = 179$)	Acc@3	80.6	25.9 [20.8, 31.0]	44.4 [39.7, 49.3]	9.4 [5.8, 13.4]	0.9 [0.3, 1.7]

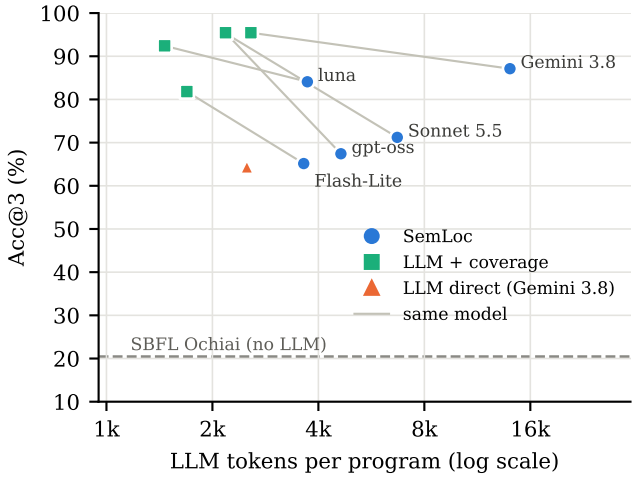


Fig. 2. Cost vs. Acc@3 on the same subset (44 programs, repaired labels); lines join the two methods of one model.

D. RQ3: Robustness

Real bugs. On the 31 youtube-dl bugs (Table V), SemLoc's top-1 is the guess by construction, and the guess's Acc@1 (46.2%) is SemLoc's; the spectrum alone places 7.5% first, close to SBFL (9.7%). LLM + coverage points the same way as on SemFault-250 (Acc@3 69.9% vs. 53.8%), but with 31 bugs from one project the CI [-3.2, +34.4] includes zero. "Fault reached" keeps the bugs where a failing test executes a ground-truth line; "artifact's own metric" is the rank the released runner reports, i.e., that of the LLM's guess.

Renamed programs. Pairing each program with its renamed copy (Table VI; Gemini 3.8 Flash only), the guess shows no detectable change (Acc@1 Δ -2.6 [-7.8, +2.4]), so we see no sign that the model guesses from memorized names. The constraint spectrum needs meaningful names: without the bonus, Acc@3 changes by -15.5 points. LLM + coverage shows no detectable change (Acc@3 Δ -0.9 [-3.5, +1.3]).

TABLE V
RQ3A: 31 YOUTUBE-DL BUGS (BUGSINPY), THREE REPEATS

	All bugs ($n = 31$)			Fault reached ($n = 26$)	
Method	Acc@1	Acc@3	Acc@5	Acc@1	Acc@3
SemLoc semantic spectrum only	7.5±1.9	29.0±3.2	37.6±3.7	9.0±2.2	34.6±3.9
SemLoc (spectrum + LLM-guess bonus)	46.2±6.7	53.8±7.4	55.9±6.7	51.3±8.0	60.3±8.9
LLM guess alone	46.2±6.7	46.2±6.7	46.2±6.7	51.3±8.0	51.3±8.0
CovGate (ours)	46.2±6.7	54.8±6.5	57.0±7.4	51.3±8.0	61.5±7.7
LLM + coverage (1 prompt, ours)	51.6±5.6	69.9±3.7	75.3±1.9	59.0±5.9	75.6±4.4
SBFL Ochiai	9.7±0.0	29.0±0.0	29.0±0.0	11.5±0.0	34.6±0.0
SBFL DStar	9.7±0.0	29.0±0.0	29.0±0.0	11.5±0.0	34.6±0.0
Artifact's own metric (rank of the LLM guess)	39.8±1.9	45.2±3.2	51.6±3.2	37.2±2.2	43.6±5.9

TABLE VI
RQ3B: ORIGINAL VS. IDENTIFIER-RENAMED PROGRAMS (REPAIRED LABELS, THREE PAIRED REPEATS)

	Acc@1			Acc@3		
Method	Original	Renamed	Δ [95% CI]	Original	Renamed	Δ [95% CI]
SemLoc	52.1±0.9	49.4±1.0	-2.6 [-7.8, 2.4]	75.3±2.0	66.8±2.0	-8.4 [-14.1, -3.0]
w/o LLM-guess bonus	42.9±2.1	34.5±2.1	-8.4 [-13.5, -3.7]	70.6±2.3	55.1±2.6	-15.5 [-21.4, -10.3]
LLM guess alone	52.1±0.9	49.4±1.0	-2.6 [-7.8, 2.4]	52.1±0.9	49.4±1.0	-2.6 [-7.8, 2.4]
LLM + coverage (1 prompt)	56.9±0.9	56.2±0.0	-0.8 [-3.6, 2.0]	92.0±0.3	91.0±1.5	-0.9 [-3.5, 1.3]

TABLE VII
RQ3C: FIVE MODELS ON THE SAME SUBSET (REPAIRED LABELS, THREE REPEATS, NO CIS)

		SemLoc		w/o LLM-guess bonus		CovGate
Model	Repeats	Acc@1	Acc@3	Acc@1	Acc@3	Acc@3
Gemini 3.8 Flash	3	52.3±2.3	87.1±1.3	53.0±4.7	84.1±0.0	87.9±2.6
GPT-6 luna	3	53.8±2.6	84.1±2.3	40.9±6.0	72.7±2.3	84.9±1.3
Claude Sonnet 5.5	3	43.2±2.3	71.2±1.3	37.9±1.3	68.9±1.3	71.2±1.3
gpt-oss-120b	3	45.5±6.0	67.4±3.5	29.5±7.9	57.6±6.6	68.2±2.3
Gemini 3.5 Flash-Lite	3	42.4±3.5	65.1±5.7	20.4±2.3	51.5±4.7	71.2±1.3

TABLE VIII
PER-PROGRAM OUTCOME CATEGORIES (REPAIRED LABELS, MAJORITY OVER THREE REPEATS); FIRST MATCH WINS

Category	Definition	Programs
gt_suspect	unverified label, every method misses top-3	6
guess_right	the LLM's guess is a ground-truth line	94
prior_hurts	spectrum ranks the fault first, the bonus lifts a wrong guess above it	19
rescued_by_spectrum	wrong guess, fault still in SemLoc's top-3	25
coverage_only_wins	SemLoc misses top-3, LLM + coverage hits it	32
all_miss	everything else	3

Model families. On the subset (Table VII; no CIs), weaker models lean harder on the guess: without the bonus, Flash-Lite's Acc@1 falls from 42.4% to 20.4% and gpt-oss-120b's from 45.5% to 29.5%. For Gemini 3.8 Flash the subset shows almost no change (52.3% vs. 53.0%), while the full test split shows a change of -9.1 points; the subset is too small to rank models on this effect.

E. Failure analysis

We categorize every test program automatically (Table VIII) and read a cluster-spread sample of up to four failures per category, nineteen in total.

The guess is right for 94 of 179 programs; the spectrum rescues 25 wrong guesses into the top three. Wrong guesses are rarely far off: over the 527 program–repeat pairs with a guess (10 runs returned none), the guess hits the fault line 281 times, lands one line below it 99 times and one line above only 28 times. In 44 of 85 wrongly guessed programs, guess and label are a compound-statement header and the statement it guards, as here, where the injected fault is the `- 1`:

```
if cap is not None and counters[key] > cap:  # label
    counters[key] = cap - 1                  # guess
```

We measure two tolerant variants: *strict* accepts the header and the first line of its body; the *upper bound* accepts the header and any line of its first body statement (up to three lines, nested guards included). SemLoc's Acc@1 rises from 52.3% to 73.9% (strict) or 76.2% (upper bound), and LLM + coverage's from 57.2% to 79.3%–81.6%; LLM + coverage stays ahead of SemLoc under every variant. Reading the sample adds three more patterns: multi-line injected blocks where either line fixes the bug, cause and symptom on one data-flow path, and fixes our oracle accepted that compensate rather than repair (e.g., subtracting one from `norm` instead of fixing `max_norm / (norm + 1)`). Most cases where the bonus "hurts" are of the first two kinds: the guess is semantically right and one line off.

VI. DISCUSSION

Reading "guessed" correctly. SemLoc's top line is the LLM's guess by design, and the failure analysis shows the guess is often semantically right and one line off the label. The lesson is therefore not that the bonus should go—on our runs removing it changes Acc@1 by -9.1 points—but that a tool

TABLE IX
CHECKLIST FOR EVALUATING LLM-BASED FAULT LOCALIZATION, WITH WHAT EACH CHECK CHANGED HERE

Check	Effect in this study
Average-rank ties	SBFL Acc@5 30.4% → 48.0%
Fixed denominator, any-hit not rank one	delta debugging 38.0% → 0.0% in the released script
Execution-verified labels	SemLoc Acc@1 36.8% → 52.3% (partly a denominator change)
Ablate every component, incl. undocumented	guess bonus: Acc@1 changes by -9.1 without it
Repeats with paired CIs	Acc@1 gain of LLM + coverage visible only when pooled over repeats
Report cost	counterfactual step: 42% of calls, no ranking change
Single prompt with the same evidence	beats the pipeline by +16.2 Acc@3 on reached faults

which claims to be grounded should state that its first answer is the model's, and that line-level labels need to accept equivalent fix locations before small differences between methods mean much.

Where execution evidence helps. Evidence earns its keep in narrowing the search: the spectrum moves the fault into the top three for 25 wrongly guessed programs, and execution evidence given directly to the model does so for 32 programs that SemLoc misses. It does not tell a right guess from a wrong one: neither a grounded bonus nor counterfactual re-ranking improves Acc@1. Its limit is reachability: when no failing test executes the faulty line, a coverage prompt cannot find it, which is why SemLoc keeps an edge at Acc@5 over all original labels. Tool designers should use execution to *shrink* the candidate set and show it to the model, and keep a fallback for unreached faults.

An evaluation checklist. Table IX collects seven checks; each changed at least one number in this study.

VII. THREATS TO VALIDITY

Internal and construct validity. Our minimal-fix oracle can accept compensating fixes and never searches alternatives for reachable labels, so the repaired labels are still noisy; we drop fixes written onto blank lines and report every result on the original labels as well. Because the oracle needs a failing test to reach the fixed line, the repaired labels favor coverage-based methods; we therefore state the main comparison on reached faults of the original labels and give the all-label numbers next to it. The oracle's proposer (GPT-6 luna, then Claude Sonnet 5.5) is also an evaluated model; the tests, not the proposer, decide, but the proposer limits which locations can be confirmed. Line-level Acc@1 depends on whether a guard or the guarded statement is labeled; we report tolerant variants. The coverage prompt is ours, fixed before seeing test results, and adds test outcomes as well as coverage, so it does not isolate coverage.

External validity. Real bugs come from one project (31 youtube-dl bugs) and the target function is chosen from the developer fix, so function-level localization is not evaluated. RQ2, the decomposition and the renaming study use one model; the model-family subset has no CIs. Models run at provider-default reasoning, so the family comparison is "at default" rather than at equal effort; the main model is newer than those in the original paper, and our re-execution with the paper's model fell short of the published logs. Renaming changes identifiers and comments but not structure, so memorization of templates is not fully excluded.

VIII. CONCLUSION AND FUTURE WORK

SemLoc's published numbers recompute exactly from its logs, but its top-1 answer is, by an undocumented design choice, the LLM's own guess; its execution evidence earns its keep at top three. On faults that a failing test reaches, handing execution evidence to the model directly—one prompt with the failing tests and per-line coverage—is more accurate, about five times cheaper and shows no detectable change under renaming; it cannot find faults no test reaches. These results come from one main model, one benchmark with repaired labels and 31 real bugs from one project. Evaluation choices and benchmark labels move the numbers as much as methods do, so we suggest that every evaluation of LLM-based fault localization apply the seven checks of Table IX. Future work includes a multi-location oracle that rejects compensating fixes, an ablation separating coverage from test outcomes in the prompt, real bugs from more projects, and function-level localization.

DATA AVAILABILITY

Code, derived data, run outputs, LLM call logs, a replayable LLM response cache and an offline Docker image are at https://github.com/doanan293/grounded-or-guessed; `REPRODUCE.md` maps every table to its script.

REFERENCES

[1] J. A. Jones, M. J. Harrold, and J. Stasko, "Visualization of test information to assist fault localization," in *Proceedings of the 24th international conference on Software engineering - ICSE '02*, ser. ICSE '02. ACM Press, 2002, p. 467. [Online]. Available: http://dx.doi.org/10.1145/581396.581397

[2] R. Abreu, P. Zoeteweij, R. Golsteijn, and A. J. van Gemund, "A practical evaluation of spectrum-based fault localization," *Journal of Systems and Software*, vol. 82, no. 11, p. 1780–1792, Nov. 2009. [Online]. Available: http://dx.doi.org/10.1016/j.jss.2009.06.035

[3] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, "A survey on software fault localization," *IEEE Transactions on Software Engineering*, vol. 42, no. 8, p. 707–740, Aug. 2016. [Online]. Available: http://dx.doi.org/10.1109/TSE.2016.2521368

[4] S. Kang, G. An, and S. Yoo, "A quantitative and qualitative evaluation of llm-based explainable fault localization," 2023. [Online]. Available: https://arxiv.org/abs/2308.05487

[5] Y. Qin, S. Wang, Y. Lou, J. Dong, K. Wang, X. Li, and X. Mao, "Agentfl: Scaling llm-based fault localization to project-level context," 2024. [Online]. Available: https://arxiv.org/abs/2403.16362

[6] C. Xu, Z. Liu, X. Ren, G. Zhang, M. Liang, and D. Lo, "Flexfl: Flexible and effective fault localization with open-source large language models," 2024. [Online]. Available: https://arxiv.org/abs/2411.10714

[7] J. Sallou, T. Durieux, and A. Panichella, "Breaking the silence: the threats of using llms in software engineering," 2023. [Online]. Available: https://arxiv.org/abs/2312.08055

[8] D. Ramos, C. Mamede, K. Jain, P. Canelas, C. Gamboa, and C. L. Goues, "Are large language models memorizing bug benchmarks?" 2024. [Online]. Available: https://arxiv.org/abs/2411.13323

[9] Z. Yang, H. Zhu, Q. Zhang, R. Gupta, and A. Kundu, "Semloc: Structured grounding of free-form llm reasoning for fault localization," 2026. [Online]. Available: https://arxiv.org/abs/2603.29109

[10] R. Widyasari, S. Q. Sim, C. Lok, H. Qi, J. Phan, Q. Tay, C. Tan, F. Wee, J. E. Tan, Y. Yieh, B. Goh, F. Thung, H. J. Kang, T. Hoang, D. Lo, and E. L. Ouh, "Bugsinpy: a database of existing bugs in python programs to enable controlled testing and debugging studies," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE '20. ACM, Nov. 2020, p. 1556–1560. [Online]. Available: http://dx.doi.org/10.1145/3368089.3417943

[11] W. E. Wong, V. Debroy, R. Gao, and Y. Li, "The dstar method for effective software fault localization," *IEEE Transactions on Reliability*, vol. 63, no. 1, p. 290–308, Mar. 2014. [Online]. Available: http://dx.doi.org/10.1109/TR.2013.2285319

[12] H. A. de Souza, M. L. Chaim, and F. Kon, "Spectrum-based software fault localization: A survey of techniques, advances, and challenges," 2016. [Online]. Available: https://arxiv.org/abs/1607.04347

[13] M. Papadakis and Y. Le Traon, "Metallaxis-fl: mutation-based fault localization," *Software Testing, Verification and Reliability*, vol. 25, no. 5-7, p. 605–628, Sep. 2013. [Online]. Available: http://dx.doi.org/10.1002/stvr.1509

[14] S. Moon, Y. Kim, M. Kim, and S. Yoo, "Ask the mutants: Mutating faulty programs for fault localization," in *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation*. IEEE, Mar. 2014, p. 153–162. [Online]. Available: http://dx.doi.org/10.1109/ICST.2014.28

[15] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, p. 183–200, Feb. 2002. [Online]. Available: http://dx.doi.org/10.1109/32.988498

[16] A. Zeller, "Isolating cause-effect chains from computer programs," *ACM SIGSOFT Software Engineering Notes*, vol. 27, no. 6, p. 1–10, Nov. 2002. [Online]. Available: http://dx.doi.org/10.1145/605466.605468

[17] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, "The daikon system for dynamic detection of likely invariants," *Science of Computer Programming*, vol. 69, no. 1-3, p. 35–45, Dec. 2007. [Online]. Available: http://dx.doi.org/10.1016/j.scico.2007.01.015

[18] M. Medeiros, U. Kulesza, C. Treude, D. Lucena, R. Gomes, R. Coelho, E. Adachi, and R. Bonifacio, "Integrating crash report mining and llms for bug localization and repair: An industrial report," 2026. [Online]. Available: https://arxiv.org/abs/2609.04483

[19] H. Yang, Y. Peng, and Z. Ding, "Effiholmes: Differential profiling-guided repository level time inefficiency fix localization," 2026. [Online]. Available: https://arxiv.org/abs/2608.03558

[20] R. Just, D. Jalali, and M. D. Ernst, "Defects4j: a database of existing faults to enable controlled testing studies for java programs," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, ser. ISSTA '14. ACM, 2014, p. 437–440. [Online]. Available: http://dx.doi.org/10.1145/2610384.2628055

[21] D. Lin, J. Koppel, A. Chen, and A. Solar-Lezama, "Quixbugs: a multi-lingual program repair benchmark set based on the quixey challenge," in *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*, ser. SPLASH '17. ACM, Oct. 2017, p. 55–56. [Online]. Available: http://dx.doi.org/10.1145/3135932.3135941

[22] S. Pearson, J. Campos, R. Just, G. Fraser, R. Abreu, M. D. Ernst, D. Pang, and B. Keller, "Evaluating and improving fault localization," in *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, May 2017, p. 609–620. [Online]. Available: http://dx.doi.org/10.1109/ICSE.2017.62

[23] C. Xu, S. Guan, D. Greene, and M.-T. Kechadi, "Benchmark data contamination of large language models: A survey," 2024. [Online]. Available: https://arxiv.org/abs/2406.04244